

Motorcycle Service Web App Analysis and Design Document : Project

**Student: Fechete Serban-Tudor
Group: 30431**

Table of Contents

Contents

1. Requirements Analysis	3
1.1 Assignment Specification	3
1.2 Functional Requirements	3
1.3 Non-functional Requirements	3
2. Use-Case Model	4
3. System Architectural Design	5

1. Requirements Analysis

1.1 Assignment Specification

The objective of this project is to provide a management system for a motorcycle service center or garage.

1.2 Functional Requirements

The application has the following functional requirements:

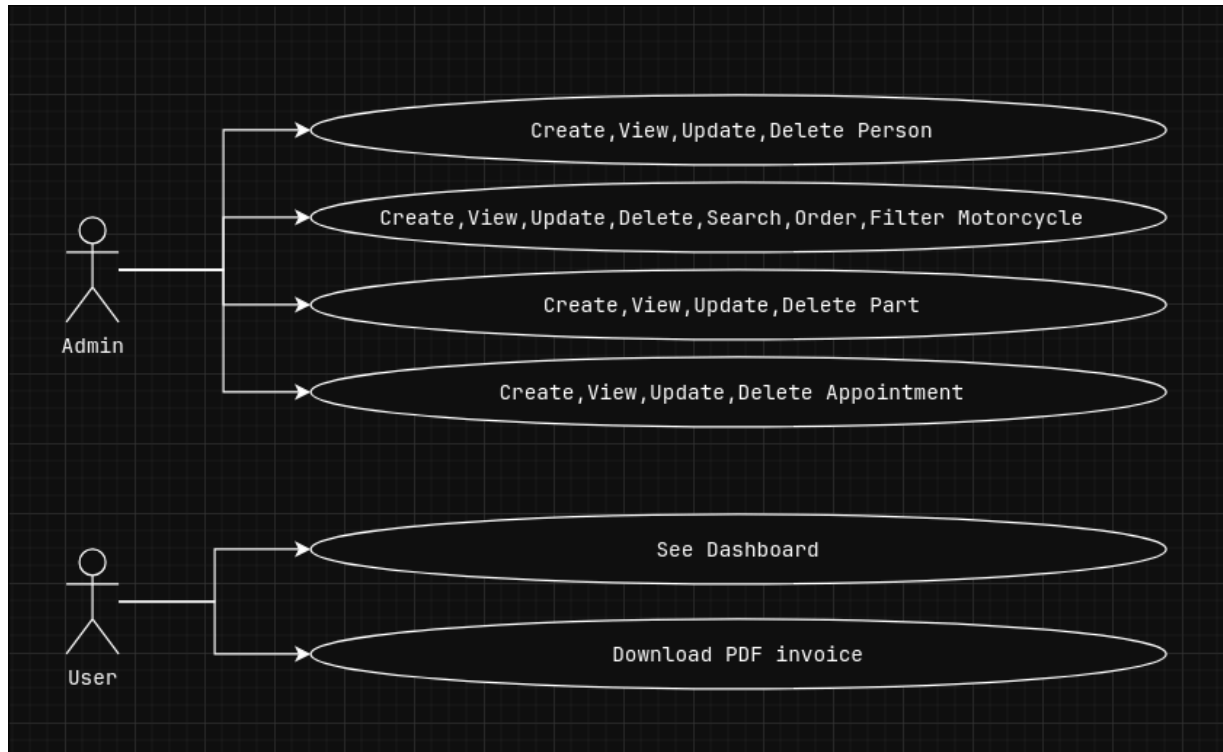
- Allow the creation and management of customer profiles (Persons), capturing details such as their name, age, password, and a unique email address.
- Allow users to register and manage multiple motorcycles associated with a specific owner, tracking the brand, model, manufacture year, and license plate.
- Allow the scheduling and tracking of service appointments for specific motorcycles.
- Allow the application to associate and track specific parts used during a given service appointment.
- Implement Role-Based Access Control to distinguish between 'Admin' (mechanics/managers) and 'Customer' (motorcycle owners), strictly restricting access to sensitive endpoints and dashboards.
- Automatically generate downloadable, structured PDF invoices for service appointments once completed, dynamically detailing labor costs and parts used.

1.3 Non-functional Requirements

The application has the following non-functional requirements:

- Use Angular to implement the frontend client interface.
- Use Java and the Spring Boot framework to build the backend REST API.
- Use PostgreSQL as the underlying relational database to store all application data.
- Use an ORM to handle database interactions, automatically update the database schema, and manage entity relationships.
- Implement database-level validation to ensure data integrity, such as enforcing non-nullable fields and unique email constraints.
- Use a layered architecture for better organization and separation of concerns.
- Secure the application against OWASP Top 10 vulnerabilities by implementing stateless JSON Web Token authentication, BCrypt password hashing, and End-to-End HTTPS/SSL encryption for data in transit.
- Containerize the entire application stack (Frontend, Backend, Database) using Docker and Docker Compose to ensure environment consistency.
- Utilize the iText7 Java library on the backend for PDF generation to prevent client-side data tampering and keep the frontend bundle size lightweight.

2. Use-Case Model



Description of the “View all motorcycles” use-case:

- Use case goal: Fetching all registered motorcycles from the system via the /motorcycle endpoint.
- Primary actor: The admin of the application.
- Main success scenario: Database connection succeeds, data is fetched correctly by the backend service, and a complete list of Motorcycle entities is returned to the client.
- Extensions:
 - Alternate scenario of success: Database connection succeeds, but no motorcycles are contained in the response because none have been registered yet.
 - Failure: Either the database connection fails, the request to /motorcycle times out, or the application encounters an internal error. The user will get a corresponding error message in the frontend.

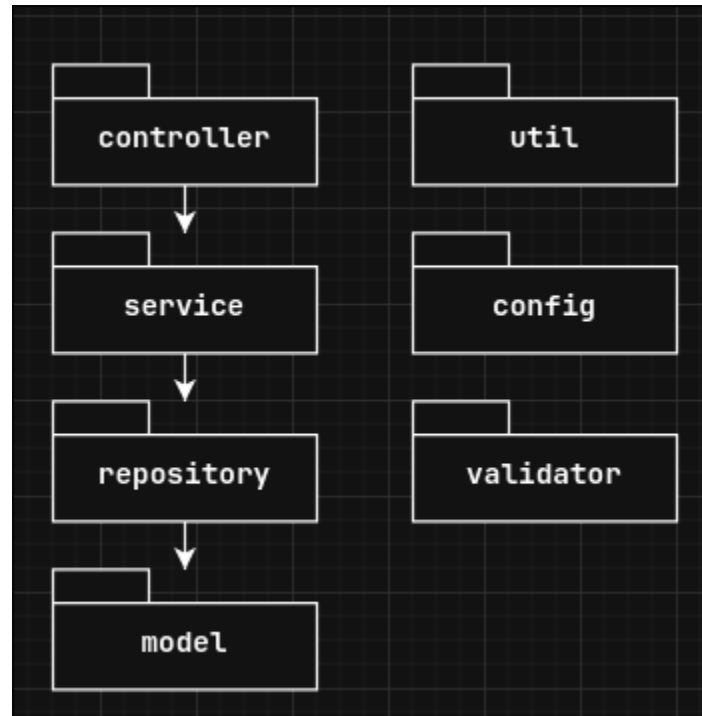
3. System Architectural Design

3.1 Architectural Pattern Description



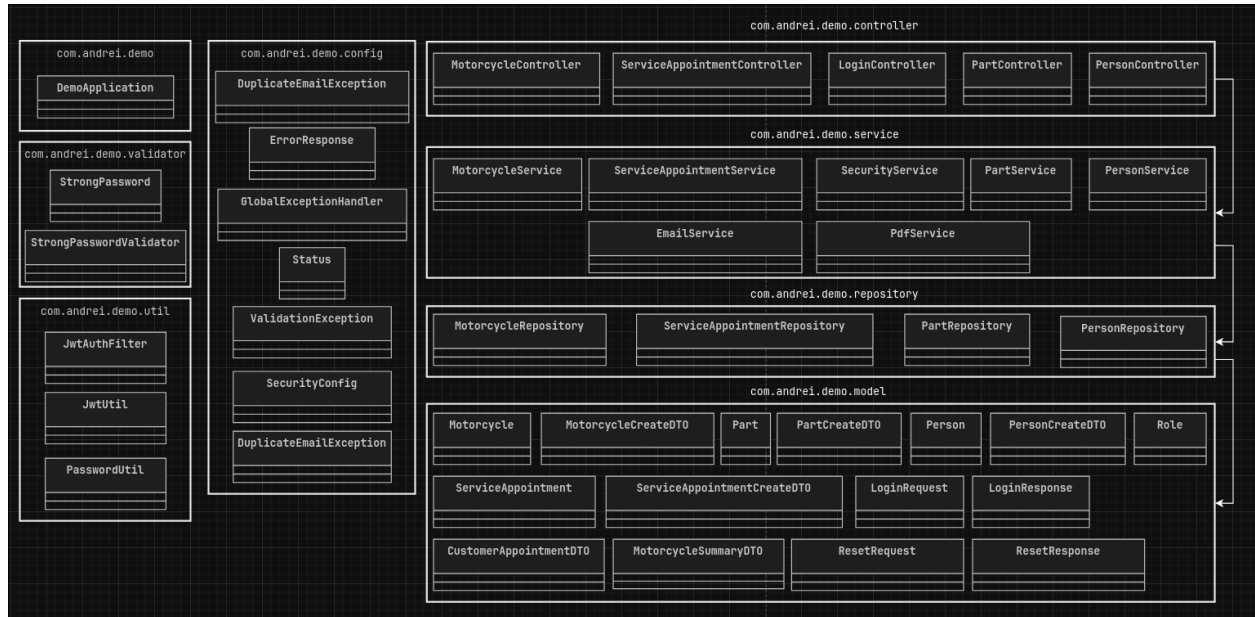
- The presentation layer handles the user interface and client-side interactions, as well as the API endpoints that expose backend functionality.
- The logic layer acts as the intermediary between the presentation and data layers, containing the core business rules and processing logic.
- The data layer is responsible for securely storing, retrieving, and managing the application's persistent data.

3.2 Package diagram

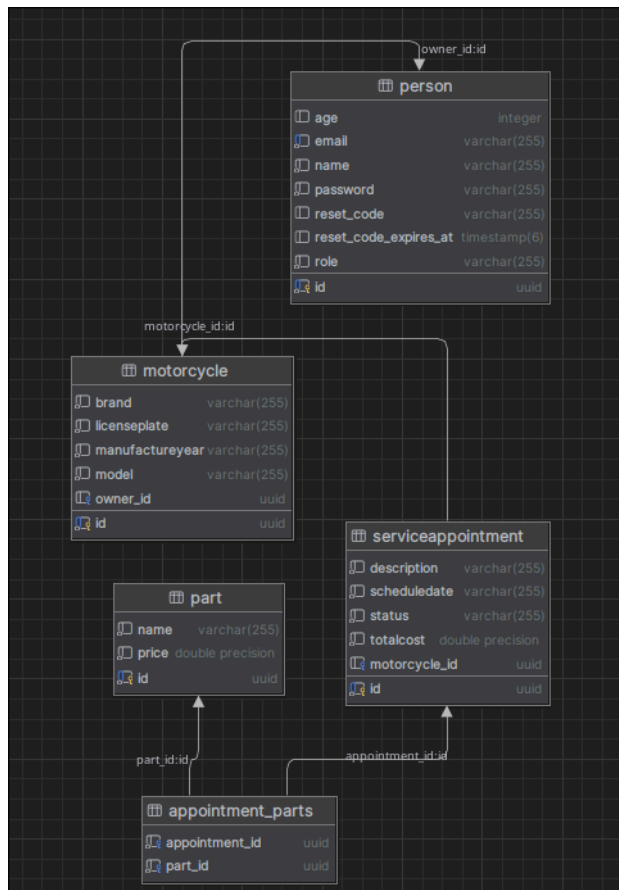


- **Controllers** : The entry point. They receive HTTP requests, trigger DTO validation, and route tasks to the Service layer without holding any business logic.
- **Services**: They enforce business rules, convert frontend DTOs into database entities, and delegate data operations to the Repository.
- **Repositories**: Manage all database interactions. Using Spring Data JPA, they automatically translate method calls into PostgreSQL queries to handle Entities.
- **Models**: The objects moving through these layers, taking two forms: DTOs and Entities.
- **Config**: Houses application-level configurations and global Spring Bean definitions. In this architecture, it strictly manages the security posture, CORS policies, and cryptographic tools like the PasswordEncoder.
- **Util**: Contains reusable, stateless helper components that support the broader application infrastructure. This includes security interceptors like the JwtAuthFilter for parsing tokens, or specialized file-handling utilities.
- **Validator**: Isolates complex, custom validation logic that goes beyond standard `@Valid` annotations. It ensures that incoming data strictly adheres to complex business constraints.

4. Class Diagram



5. Data Model



6. Changes for the Project

1. Containerization with Docker

To ensure environment consistency and seamless deployment, I containerized the entire application stack.

- Design Decision: I migrated from a local PostgreSQL and Node/Java setup to a multi-container Docker Compose architecture (Frontend, Backend, Database). I chose this approach to eliminate the "it works on my machine" problem.

2. Comprehensive Security Implementation

Beyond standard CRUD operations, I hardened the application architecture against the OWASP Top 10 vulnerabilities. Instead of relying on default configurations, I implemented the following defense-in-depth measures:

- Authentication & Access Control: I replaced stateful session cookies with a stateless, cryptographically signed JWT authentication system. I strictly enforced Role-Based Access Control using `@PreAuthorize` on the backend and Angular Route Guards (with role-bouncing) on the frontend. To prevent username enumeration, the login system was designed to return generic error messages.
- Data Protection & Architecture: Data in transit is protected via End-to-End SSL/HTTPS encryption (using a custom Java keytool certificate), while Data at rest is secured by hashing all passwords with `BCryptPasswordEncoder`. I utilized the DTO pattern to completely hide database entities from the client and applied `@Valid` annotations to sanitize all incoming data, preventing Injection attacks.
- Infrastructure Resilience & Auditing: I locked down the server with a strict CORS policy. To prevent information leakage during server crashes, I built a `@RestControllerAdvice` Global Exception Handler that hides raw Java stack traces and returns safe, generic JSON errors. Finally, I added Slf4j logging for security events and utilized the OWASP Dependency-Check Maven plugin to forcefully patch vulnerable third-party libraries.

3. Non-CRUD Feature:

To bridge the gap between digital data and physical business operations, I implemented an automated PDF generation engine using the `iText7` Java library.

- Design Decision: Generating PDFs on the backend ensures that pricing logic and formatting cannot be tampered with by the client. The backend returns a binary Blob over a `@Transactional` connection, which the Angular frontend dynamically downloads into the user's local file system.